SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY
An Autonomous Institution |Approved by AICTE | Affiliated to Anna University |
Accredited by NAAC with A++ Grade
Kuniamuthur, Coimbatore – 641008.

QUIZ APPLICATION MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

SAMEETH P

In partial fulfillment for the award of the degree

of

Bachelor of Engineering
In
Computer Science & Engineering

SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY

An Autonomous Institution |Approved by AICTE | Affiliated to Anna University |

Accredited by NAAC with A++ Grade

Kuniamuthur, Coimbatore – 641008.

SEPTEMBER 2026

SRI KRISHNA COLLEGE OF ENGINEERING AND TECHNOLOGY
An Autonomous Institution |Approved by AICTE | Affiliated to Anna University |
Accredited by NAAC with A++ Grade
Kuniamuthur, Coimbatore – 641008.

BONAFIDE CERTIFICATE

Certified that this project report **"QUIZ APPLICATION MANAGEMENT SYSTEM"** is the bonafide work of "**SAMEETH P (2403727710421222)**" who carried out the project work under my supervision.

Signature of the Head of the Department	**Signature of the Supervisor**
Dr. Kousika	**Ms. Nandhini N**
HEAD OF THE DEPARTMENT	**SUPERVISOR**
Assosiate Professor & Head **Department of Computer Science and Engineering**	**Assistant Professor** **Department of Computer Science and Engineering**
Kuniyamuthur, Coimbatore – 641008	**Kuniyamuthur, Coimbatore – 641008**

Submitted for the Project viva-voce examination held on_____

INTERNAL EXAMINER EXTERNAL EXAMINER

ABSTRACT

The **Quiz Management System** is a web-based application developed to automate and simplify the process of creating, managing, and conducting online quizzes. It provides separate functionalities for **Administrators, Educators, and Students** through role-based access control. Administrators can manage users and monitor quiz activities, while educators can create, update, and manage quizzes and questions. Students can securely access available quizzes, participate in assessments, submit answers, and view their results. The system supports **multiple-choice questions, automated evaluation, score calculation, and instant result generation**, reducing manual effort and evaluation errors. Quiz attempts and student responses are stored in a centralized database for efficient management and future reference. The system also provides **analytics and gradebook management** to monitor student performance and maintain assessment records. The frontend is developed using **React.js**, while **Spring Boot** is used for backend development and RESTful API communication. **MySQL** is used for storing and managing application data, while **Spring Data JPA and Hibernate** handle database operations. **Spring Security with JWT** provides secure authentication and role-based authorization for protected resources. **Swagger/OpenAPI** is integrated for API documentation and testing. The layered **Controller, Service, Repository, and Entity** architecture improves maintainability, scalability, and separation of responsibilities. Overall, the system provides a secure, efficient, centralized, and user-friendly platform for conducting online assessments while improving quiz management, evaluation accuracy, and performance tracking.

LIST OF FIGURES

Figure No.	Description	Page No.
Figure 1.1	Three-Tier Architecture	12
Figure 1.2	Entity Relationship Diagram	15
Figure 1.3	Class Diagram	16
Figure 1.4	Activity Diagram	17
Figure 1.5	Administrator Sequence Diagram	18
Figure 1.6	Educator Sequence Diagram	18
Figure 1.7	Student Sequence Diagram	19
Figure 1.8	Swagger Implementation	20
Figure 1.9	Backend Folder Structure	21
Figure 1.10	React Tree	22
Figure 1.11	Frontend UI(Dashboard)	23
Figure 1.12	Admin Dashboard Page	23
Figure 1.13	Educator Dashboard Page	24
Figure 1.14	Student Dashboard Page	24
Figure 1.15	Frontend Folder Structure	26
Figure 1.16	Testing	30

LIST OF SYMBOLS, ABBREVIATIONS AND NOMENCLATURE

Term	Definition / Full Form
API	Application Programming Interface
REST	Representational State Transfer
UI	User Interface
SRS	Software Requirements Specification
CORS	Cross-Origin Resource Sharing
JPA	Java Persistence API
JWT	JSON Web Token
RBAC	Role-Based Access Control
CRUD	Create, Read, Update, Delete
AI	Artificial Intelligence
React.js	JavaScript Library for Building User Interfaces
HTML	HyperText Markup Language
CSS	Cascading Style Sheets
JS	JavaScript
JSX	JavaScript XML
DOM	Document Object Model
HTTP	HyperText Transfer Protocol
JSON	JavaScript Object Notation
Axios	JavaScript Library for Making HTTP Requests
npm	Node Package Manager
Responsive UI	User Interface that adapts to different screen sizes

TABLE OF CONTENTS

ABSTRACT ……	3
LIST OF FIGURES……	4
LIST OF SYMBOLS, ABBREVIATIONS, AND NOMENCLATURE…....	4
1. INTRODUCTION…….....	8
1.1 General Overview……..	8
1.2 Purpose and Objectives……..	8
1.3 Product Scope…….....	8
2. OVERALL DESCRIPTION…….....	10
2.1 Product Perspective and Functions…….....	10
2.2 User Classes and Characteristics…….....	10
2.3 Operating Environment and Design Constraints…….....	11
2.4 External Interface Requirements……	11
3. SYSTEM FUNCTIONAL REQUIREMENTS……	12
3.1 Functional Requirement Table…….....	12
3.2 System Requirements……..	13
3.2.1 Hardware Requirements…….	13
3.2.2 Software Requirements……..	13
4. SYSTEM DESIGN……..	15
4.1 Presentation Layer – React.js Frontend…….	15
4.2 Business Logic Layer – Spring Boot Backend……...	16
4.3 Data Layer – MySQL Database…….....	16
4.4 UML Diagrams…….....	17
4.4.1 Entity Relationship Diagram……	17

4.4.2 Class Diagram……..	18
4.4.3 Activity Diagram……..	19
4.4.4 Sequence Diagram……	20
5. IMPLEMENTATION……	25
5.1 Backend Implementation…….	25
5.1.1 Spring Boot Backend and REST API Implementation……..	25
5.1.2 Backend Security and Business Logic Implementation…….	25
5.2 Frontend Implementation……	25
5.2.1 React.js User Interface and Component Implementation……..	25
5.2.2 Frontend–Backend Integration and State Management…….	25
6.TECHNICAL DESIGN AND SYSTEM ATTRIBUTES…….....	26
6.1 Performance and Reliability Targets…….....	26
6.2 REST API Reference (Endpoints)……...	27
6.2.1 Quiz Management…….	27
6.2.2 Question Management……..	28
6.2.3 Quiz Attempt and Response Management……..	28
6.2.4 User Authentication and Administration…….	29
6.2.5 Performance and Platform Analytics……...	29
7. TESTING……..	30
8. CONCLUSION & FUTURE ENHANCEMENTS…… ……	30
9.APPENDIX : Role-Based Permission Matrix……	

1. INTRODUCTION

1.1 General Overview

Online assessment systems are essential for modern education and training. The **Quiz Management System** provides a web-based solution for creating and conducting quizzes efficiently. It automates question management, quiz attempts, response tracking, score calculation, and performance analysis. The system uses **React.js, Spring Boot, and MySQL** for full-stack development. **JWT authentication, Spring Security, and role-based authorization** ensure secure access to the application. Overall, the system reduces manual effort, improves assessment efficiency, and provides secure management of quiz and performance data.

1.2 Purpose and Objectives

The primary purpose of this document is to define the comprehensive requirements and architectural blueprint for the **Quiz Management System**. The system is designed to meet three core objectives:

- **Automation of Assessment:** Automating quiz creation, question management, answer evaluation, and result generation.
- **Efficient Performance Tracking:** Providing score calculation, result management, analytics, and gradebook functionality to monitor student performance.
- **Secure Quiz Management:** Implementing JWT authentication and role-based access control to provide secure access for Administrators, Educators, and Students.

1.3 Product Scope

The **Quiz Management System** provides a comprehensive platform for online assessment, characterized by seven key functional areas:

- **User Authentication:** Secure login and registration with JWT-based authentication and role-based access control.
- **Quiz Management:** Creation, viewing, updating, deletion, and organization of quizzes by authorized users.
- **Question Management:** Adding, editing, deleting, and managing questions, MCQ options, correct answers, and marks.
- **Online Quiz Attempts:** Allows students to access quizzes, answer questions, submit responses, and complete assessments.
- **Automated Evaluation:** Automatically evaluates submitted answers, calculates scores, and generates results.
- **Analytics and Gradebook:** Tracks student performance, quiz scores, averages, and maintains assessment records.
- **Secure API Management:** Provides protected REST APIs using Spring Security and JWT, with Swagger/OpenAPI for documentation and testing.

2. OVERALL DESCRIPTION

2.1 Product Perspective and Functions

The platform operates as a web-based system built on a layered architecture. The **Spring Boot backend** provides business logic and RESTful API services, while the **React.js frontend** provides the user interface. The architecture supports the following primary functions:

- **Role-based user authentication** using JWT and Spring Security.
- **Quiz management** for creating, updating, viewing, and deleting quizzes.
- **Question management** for adding questions, MCQ options, correct answers, and marks.
- **Online quiz attempts** for students to access quizzes and submit answers.
- **Automated evaluation and result generation** for calculating scores and displaying results.
- **Response management** for storing and retrieving student answers.
- **Analytics and gradebook management** for tracking quiz scores and student performance.
- **REST API integration** between the React.js frontend, Spring Boot backend, and MySQL database, with Swagger/OpenAPI for API documentation and testing.

2.2 User Classes and Characteristics

The system implements a role-based access structure to ensure secure and controlled access to different functionalities:

- **Administrator:** Manages users, quizzes, questions, system activities, analytics, and gradebook records.
- **Educator:** Creates and manages quizzes and questions and monitors student performance.
- **Student:** Accesses available quizzes, submits answers, views results, and tracks performance.

2.3 Operating Environment and Design Constraints – Technology Used

- **Backend Technology**: Java 17+ with Spring Boot 3.x for developing backend services and REST APIs.
- **Frontend Technology**: React.js for developing the interactive web-based user interface.
- **Database Technology:** MySQL with Spring Data JPA/Hibernate for persistent data storage.
- **Authentication & Security:** Spring Security with JWT for secure authentication, authorization, and role-based access control (RBAC).
- **API Communication:** REST APIs using JSON for communication between the React.js frontend and Spring Boot backend.

2.4 External Interface Requirements

The system communicates between the frontend, backend, and database through the following technologies:

- **REST APIs:** Used for communication between the React.js frontend and Spring Boot backend.
- **Database Integration:** MySQL is connected through Spring Data JPA/Hibernate for storing and retrieving application data.
- **API Data Format:** JSON is used for exchanging data between frontend and backend.
- **Secure Communication:** Protected APIs use JWT authentication and Spring Security for secure access.
- **API Documentation:** Swagger/OpenAPI is used for documenting and testing backend REST APIs.

3. SYSTEM FUNCTIONAL REQUIREMENTS

3.1 Functional Requirement Table

FRID	Functional Requirement	Description
FR1	**User Registration and Authentication**	The system shall allow users to register and securely log in using JWT authentication.
FR2	**Role-Based Access Control**	The system shall provide different access permissions for Administrators, Educators, and Students.
FR3	**Quiz Management**	The system shall allow authorized users to create, view, update, and delete quizzes.
FR4	**Question Management**	The system shall allow authorized users to add, view, update, and delete quiz questions and MCQ options.
FR5	**Quiz Attempt Management**	The system shall allow students to start quizzes, answer questions, submit attempts, and complete quizzes.
FR6	**Automated Evaluation and Results**	The system shall automatically evaluate submitted answers, calculate scores, and generate results.
FR7	**Response Management**	The system shall store and retrieve student responses associated with questions and quiz attempts.
FR8	**Analytics and Performance Tracking**	The system shall calculate performance statistics such as average, highest, and lowest scores.
FR9	**Gradebook Management**	The system shall store and display student grades and quiz performance records.
FR10	**Secure API and Documentation**	The system shall provide protected REST APIs using Spring Security and JWT, with Swagger/OpenAPI for API documentation and testing.

3.2. SYSTEM REQUIREMENTS

3.2.1 Hardware Requirements :

Component	Minimum Requirement
Processor	Intel Core i5 /AMD Ryzen 5 series
RAM	8 GB
Network	Broadband Internet Connection

3.2.3 Software Requirements:

Software	Version / Requirement
Operating System	Windows 10/11, Ubuntu 22.04 LTS, or macOS
Java Development Kit (JDK)	JDK 17 or later
Integrated Development Environment (IDE)	IntelliJ IDEA / Eclipse IDE / Visual Studio Code
MySQL Server	MySQL 8.0 or later
Apache Maven	Maven 3.8 or later
Git	Latest Stable Version
Web Browser	Google Chrome / Microsoft Edge / Safari

4. SYSTEM DESIGN

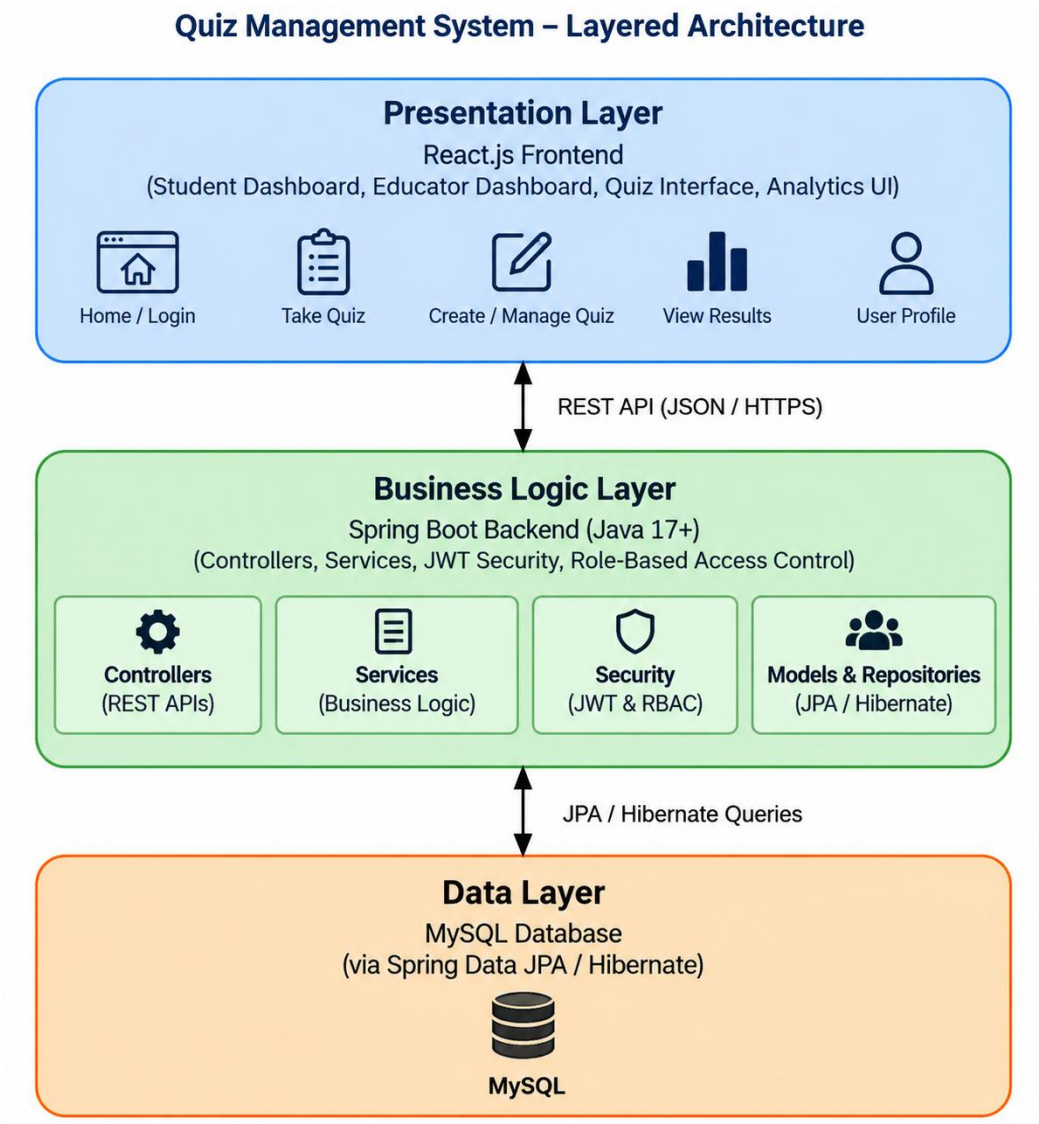


Figure 1.1-Three Tier Architecture

Three-Tier Arch

4.3.1 Presentation Layer – React.js Frontend

This Layer provides the user interface for administrators, educators, and students. It handles **user interaction, navigation, quiz management, question display, quiz attempts, and result presentation**. The frontend communicates with the backend using **Axios and REST APIs**.

4.3.2 Business Logic Layer – Spring Boot Backend

This is the core processing layer of your application. It is developed using Java and Spring Boot and contains controllers, services, security components, and business logic. The controller receives requests from the React frontend, the service layer processes the required operation, and security mechanisms such as JWT authentication and RBAC verify whether the user is authorized to perform the operation before accessing or modifying the data.

4.3.3 Data Layer – MySQL Database

Manages persistent application data using **Spring Data JPA and Hibernate**. It maps Java entities to MySQL tables and performs **CRUD operations** for entities such as **Users, Quizzes, Questions, Quiz Attempts, and Responses**, ensuring reliable data storage and retrieval.

4.4 UML DIAGRAMS:

4.4.1 Entity – Relationship:

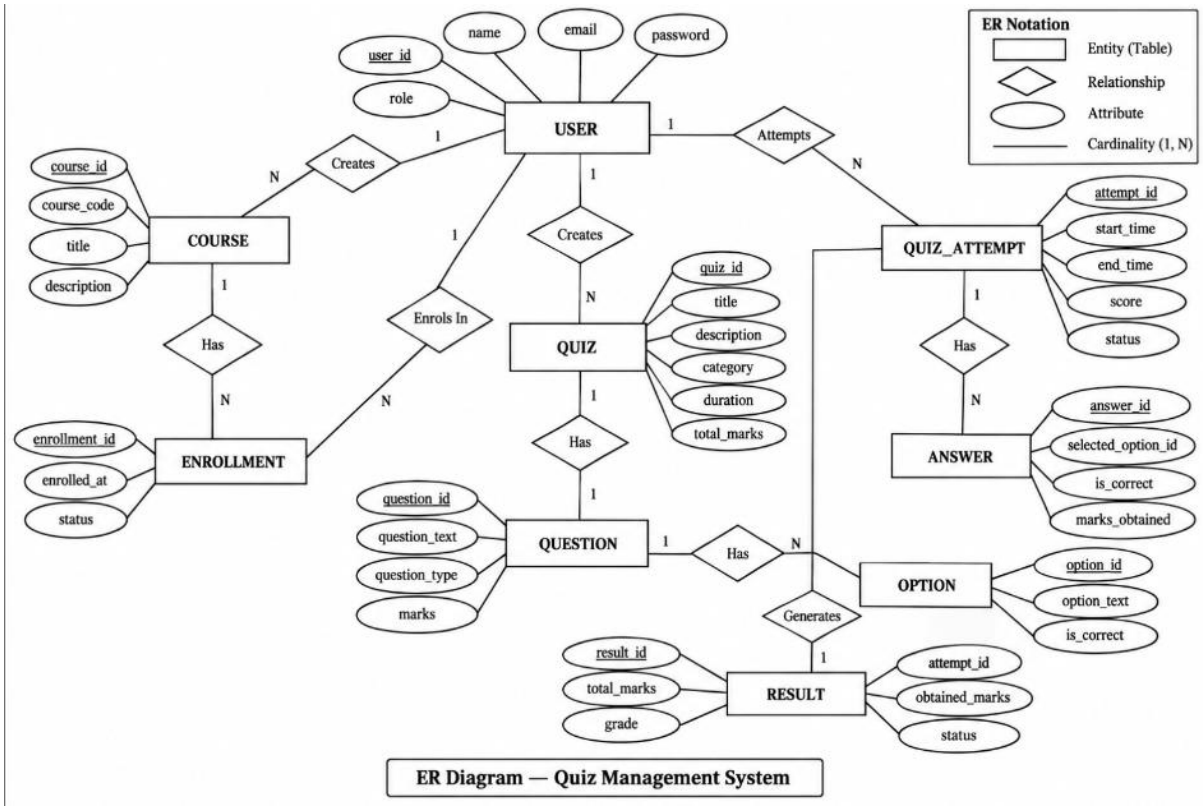


Figure 1.2 –Entity Relationship Diagram

Entity Relationship Symbols

Symbol	Name
Entity	Entity (Table)
	Relationship
Attribute	Attribute
id	Primary Key Attribute
	Connection Line
1 N	Cardinality (1 : N)

4.4.2. CLASS DIAGRAM:

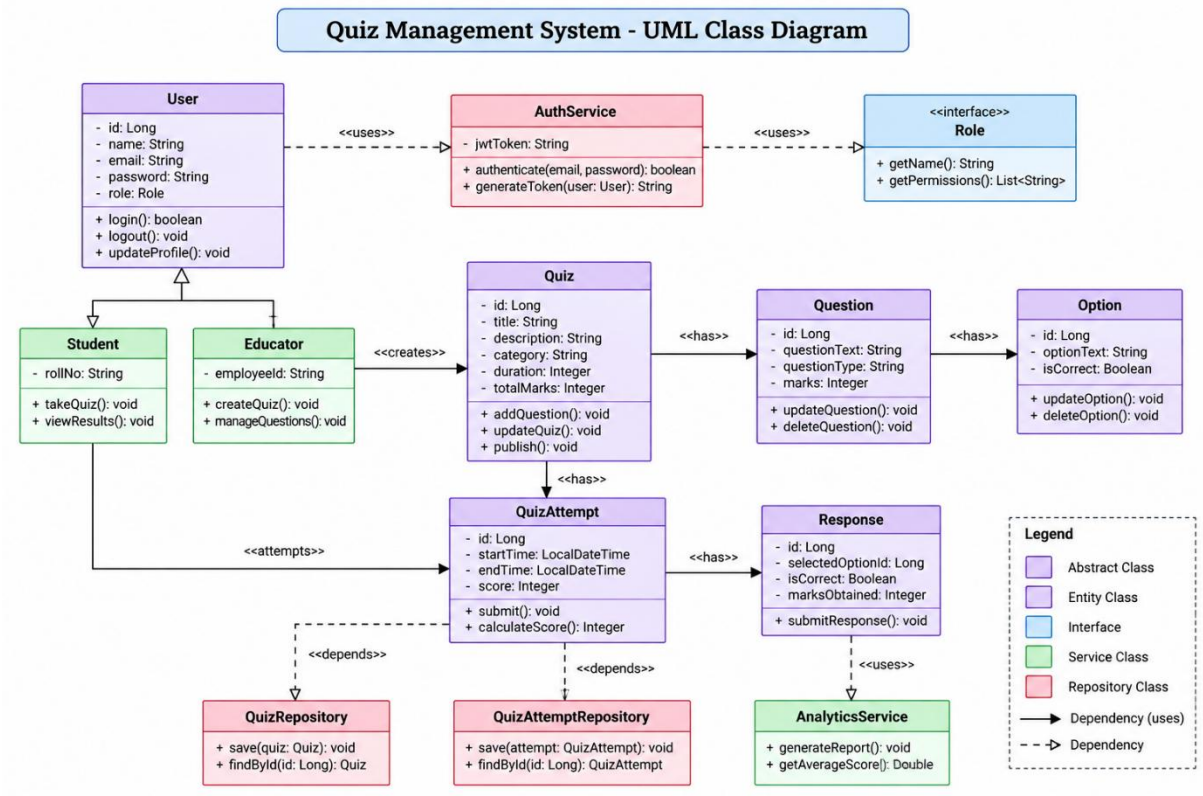
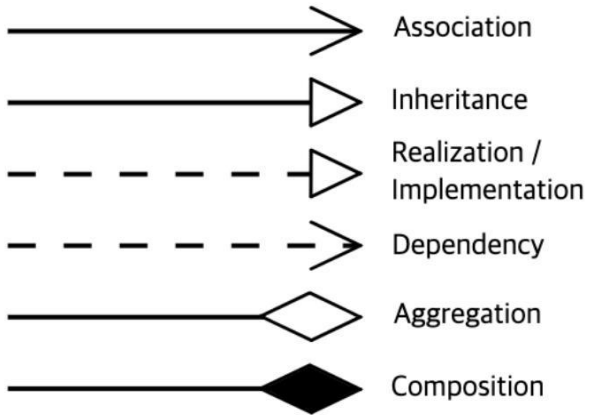


Figure 1.3 – Class Diagram

CLASS RELATIONSHIPS:



4.4.3. ACTIVITY DIAGRAM:

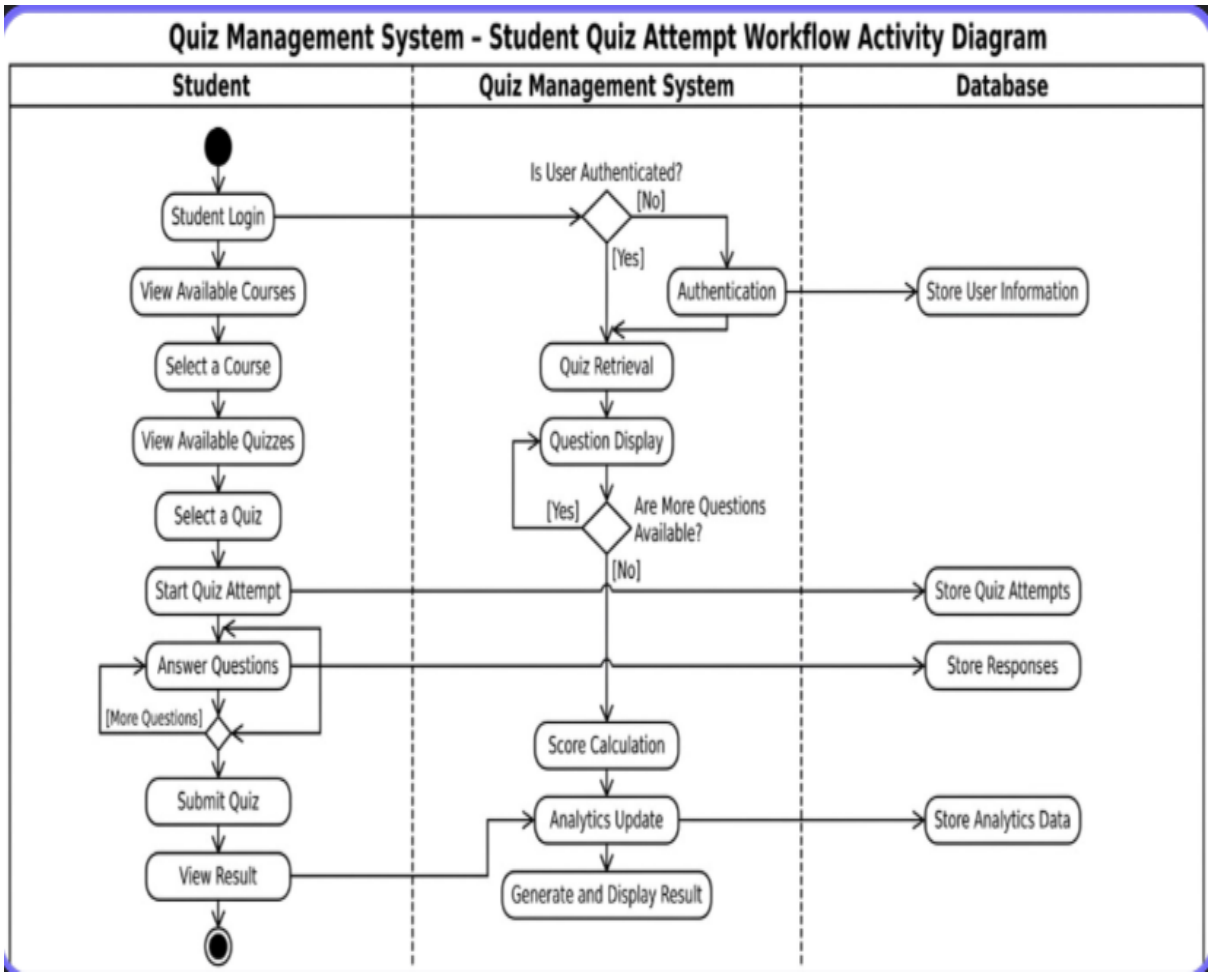


Figure 1.4 – Activity Diagram

4.4.4. SEQUENCE DIAGRAM:

I) ADMINISTRATOR

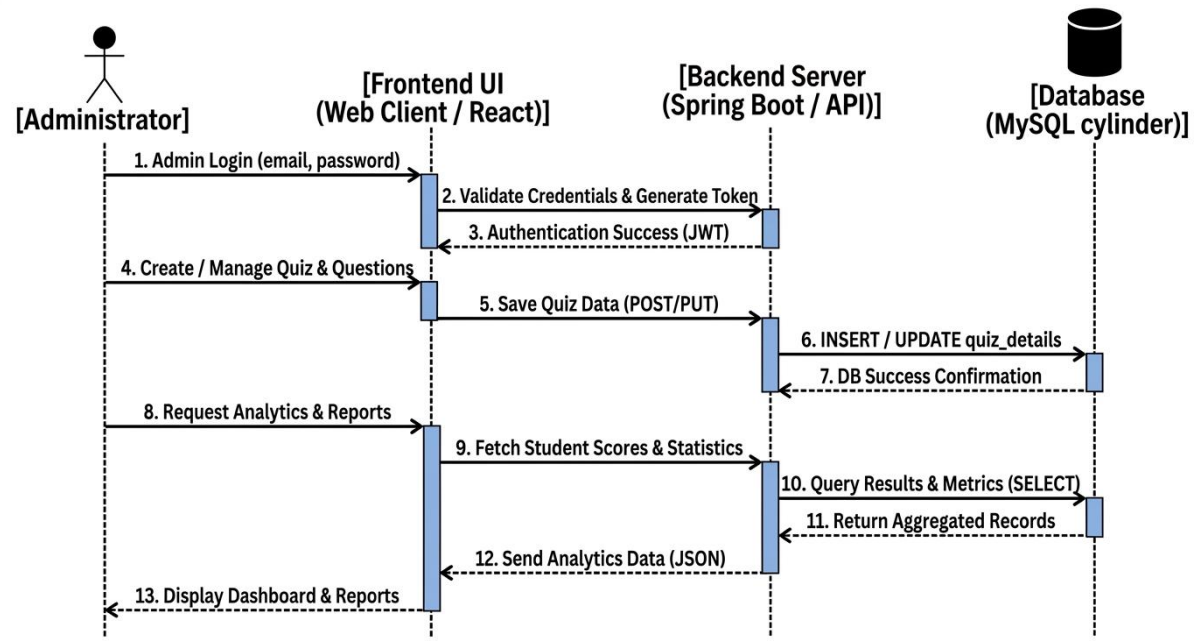


Figure 1.5 Sequence Diagram(Admin)

2)EDUCATOR

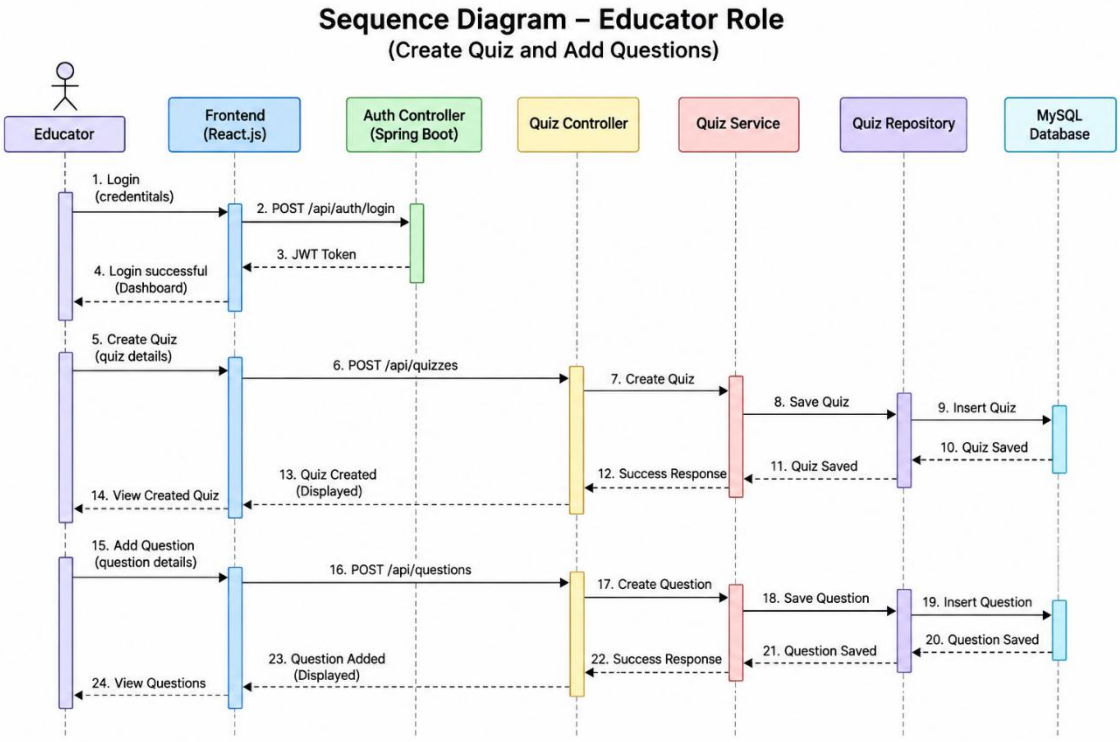


Figure 1.6 Sequence Diagram(Educator)

3) STUDENT

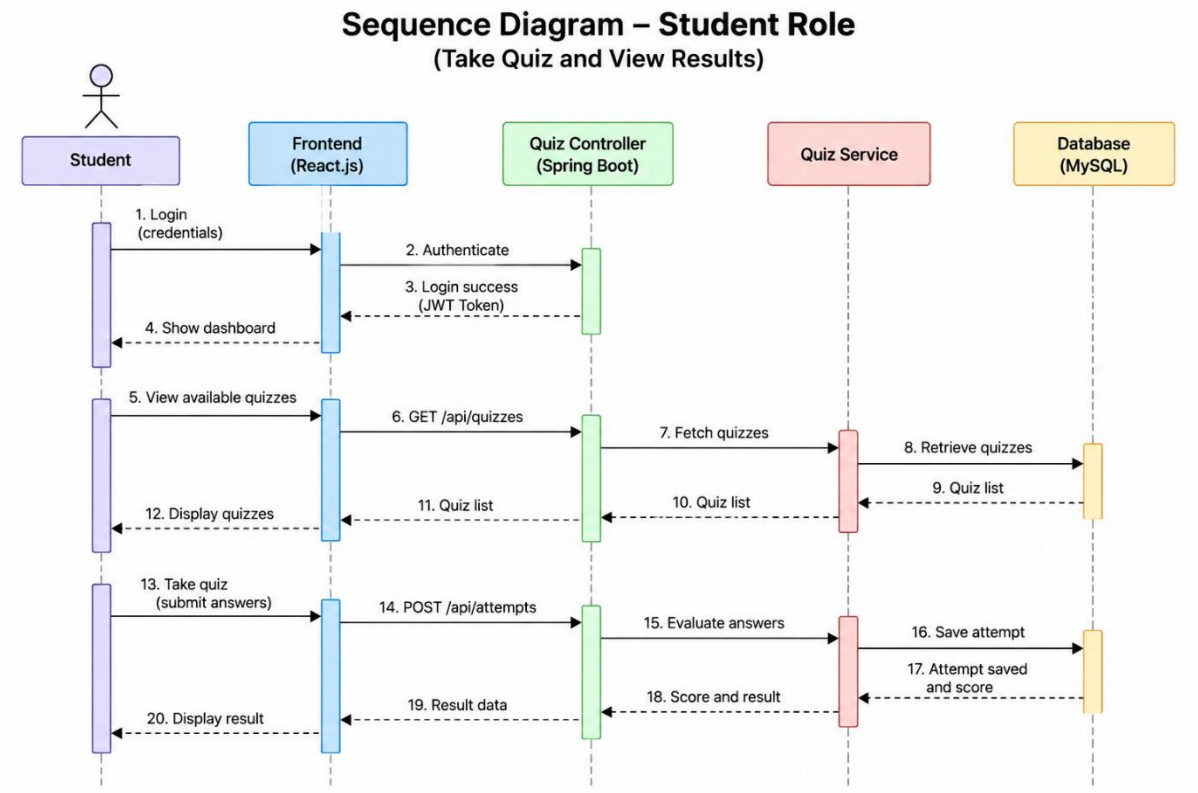


Figure 1.7 Sequence Diagram(Student)

5. IMPLEMENTATION

5.1 Backend Implementation

5.1.1 Spring Boot Backend and REST API Implementation

The backend is implemented using Java 17+ and Spring Boot, following a layered architecture consisting of controllers, services, repositories, and entity classes, where REST APIs handle chatbot management, conversation processing, analytics, and enterprise integration operations while exchanging data with the frontend through JSON.

5.1.2 Backend Security and Business Logic Implementation

The backend implements Spring Security, JWT-based authentication, role-based access control, input validation, business rules, exception handling, CORS configuration, and database operations using Spring Data JPA/Hibernate, ensuring that platform resources and chatbot functionalities are accessed only by authorized users.

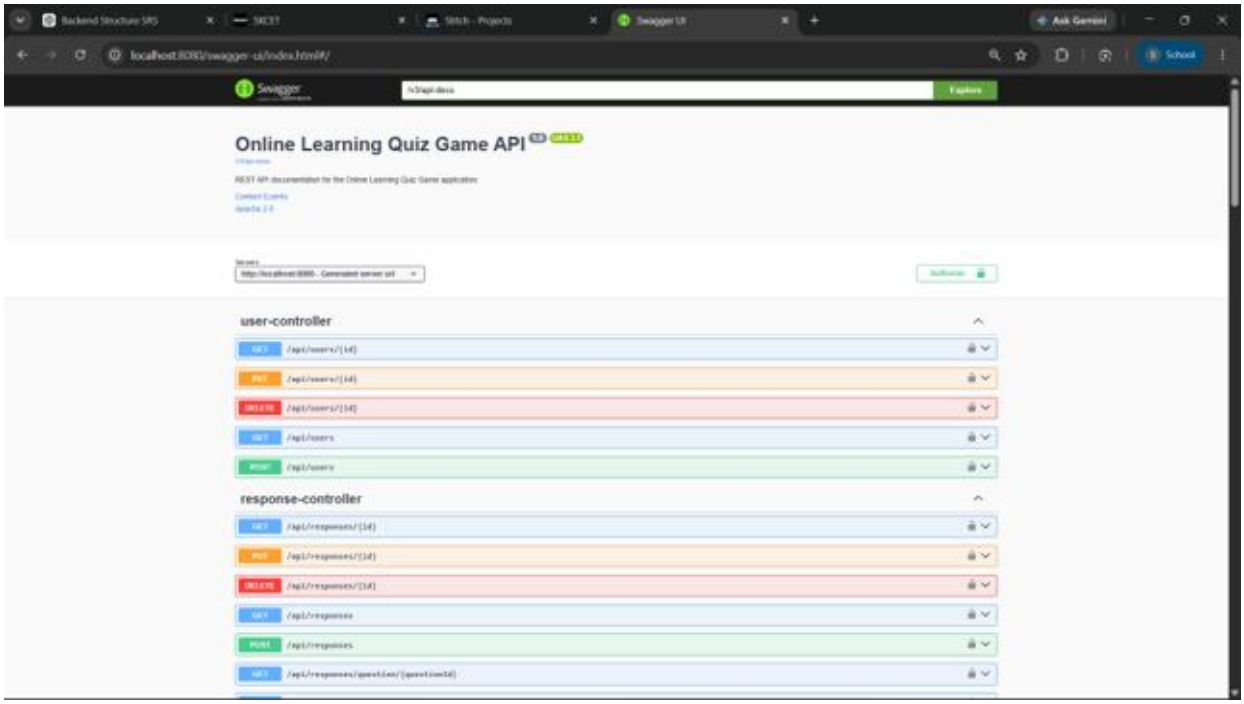


Figure 1.8 – Swagger UI

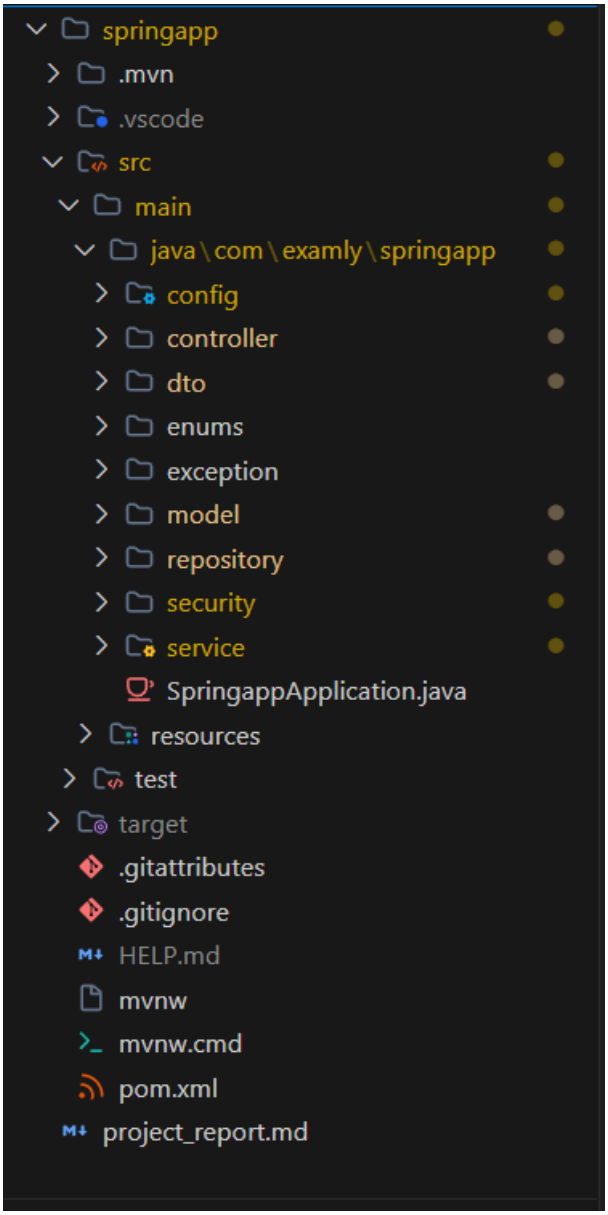


Fig 1.9 Backend Folder Structure

5.2 Frontend Implementation

5.2.1 React.js User Interface and Component Implementation

The frontend is implemented using **React.js with reusable JSX components** to provide an interactive **quiz management interface** containing dashboards, quiz management, question management, online quiz attempts, result display, analytics, gradebook management, navigation, forms, tables, and responsive layouts for different user roles.

5.2.2 Frontend–Backend Integration and State Management

The React frontend communicates with the Spring Boot backend using Axios and REST APIs, performing GET, POST, PUT, and DELETE operations while managing application state, form data, authentication responses, loading states, validation messages, API errors, and dynamically updating the user interface based on backend responses.

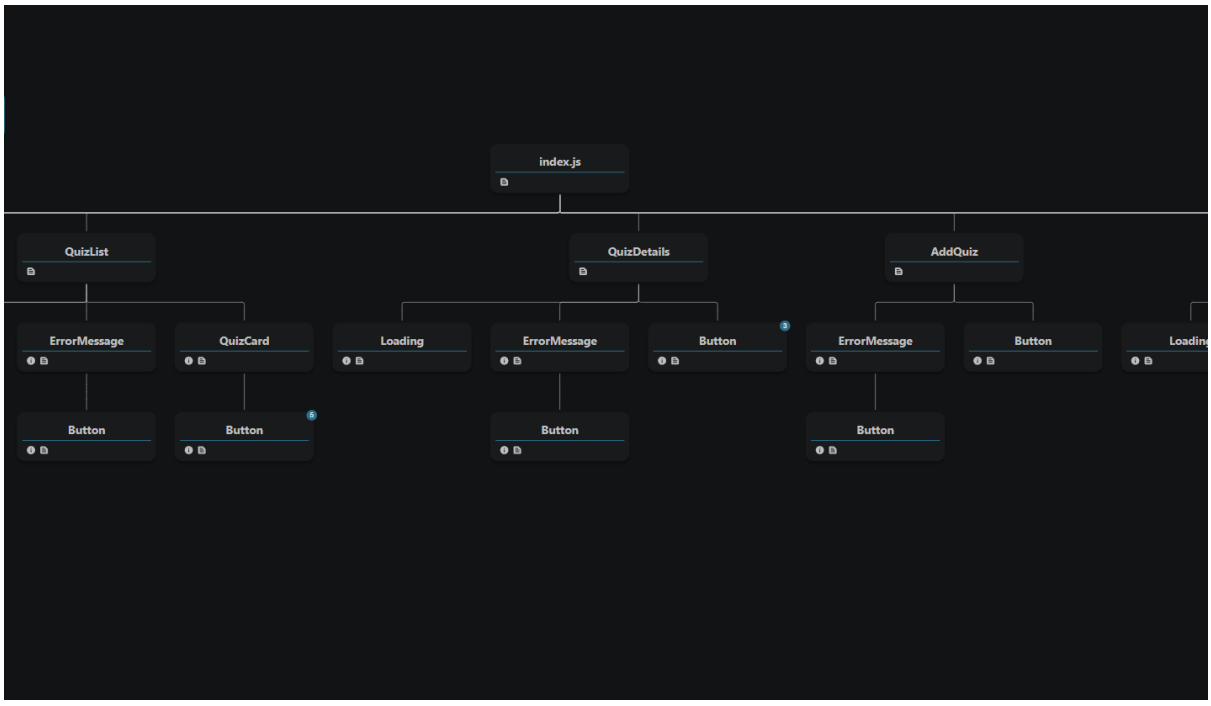


Figure 1.10 React Tree

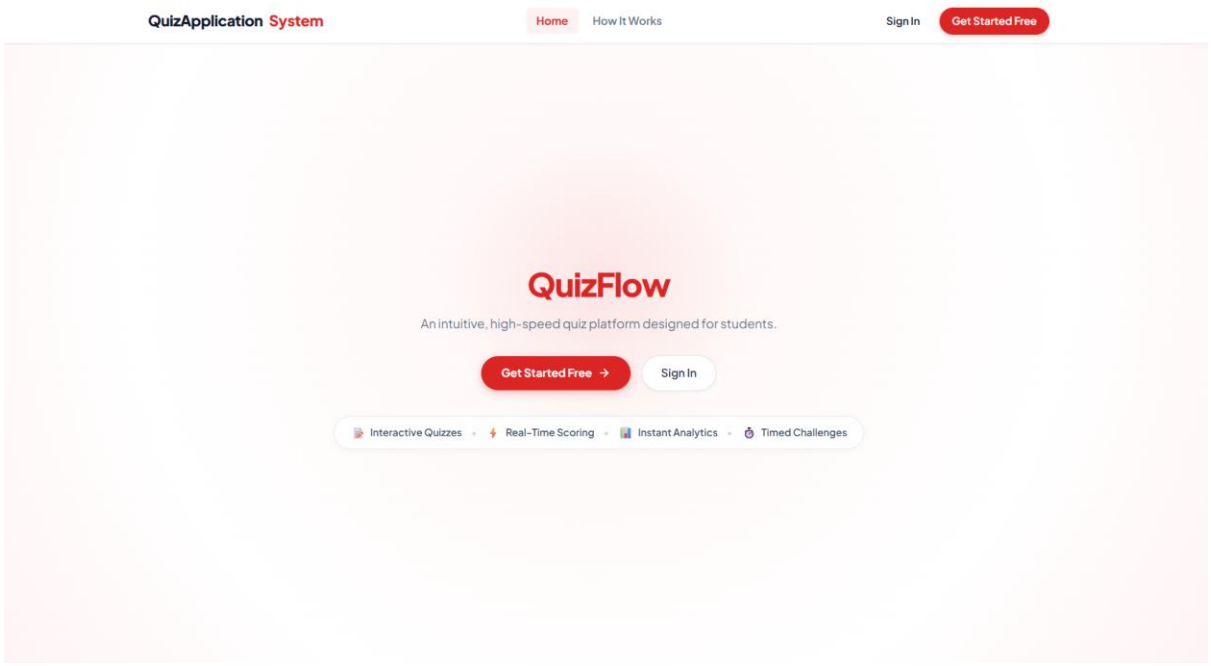


Figure 1.11 Frontend UI(Dashboard)

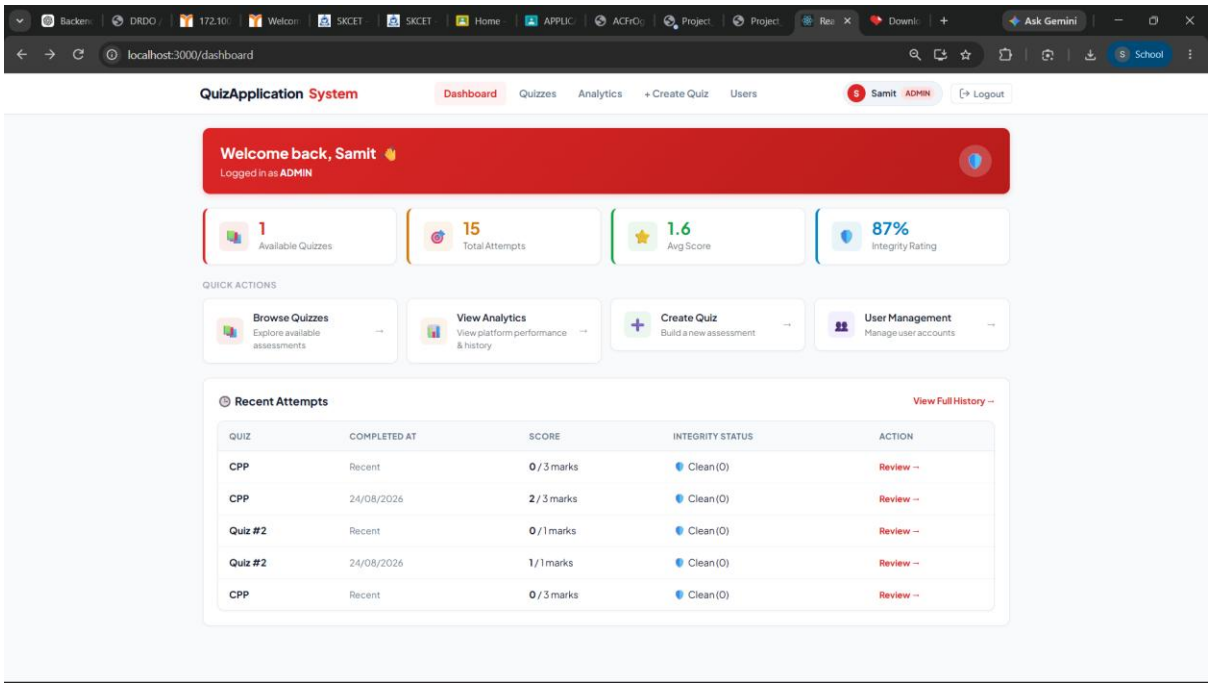


Figure 1.12 Admin Dashboard

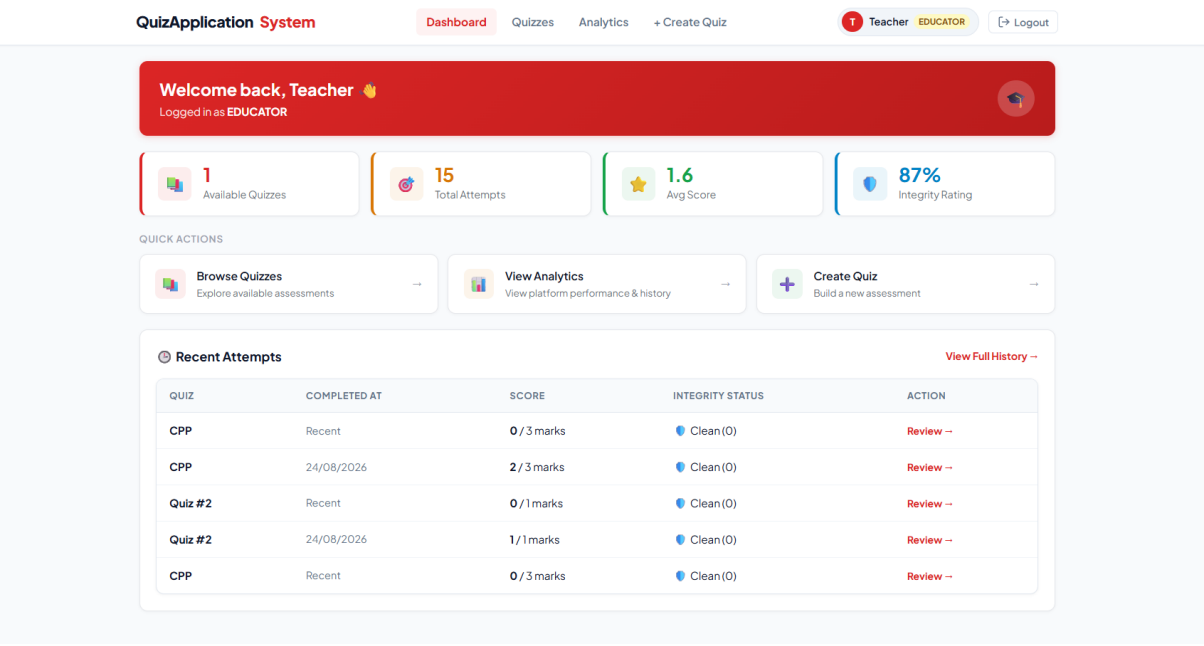


Figure 1.13 Educator Dashboard

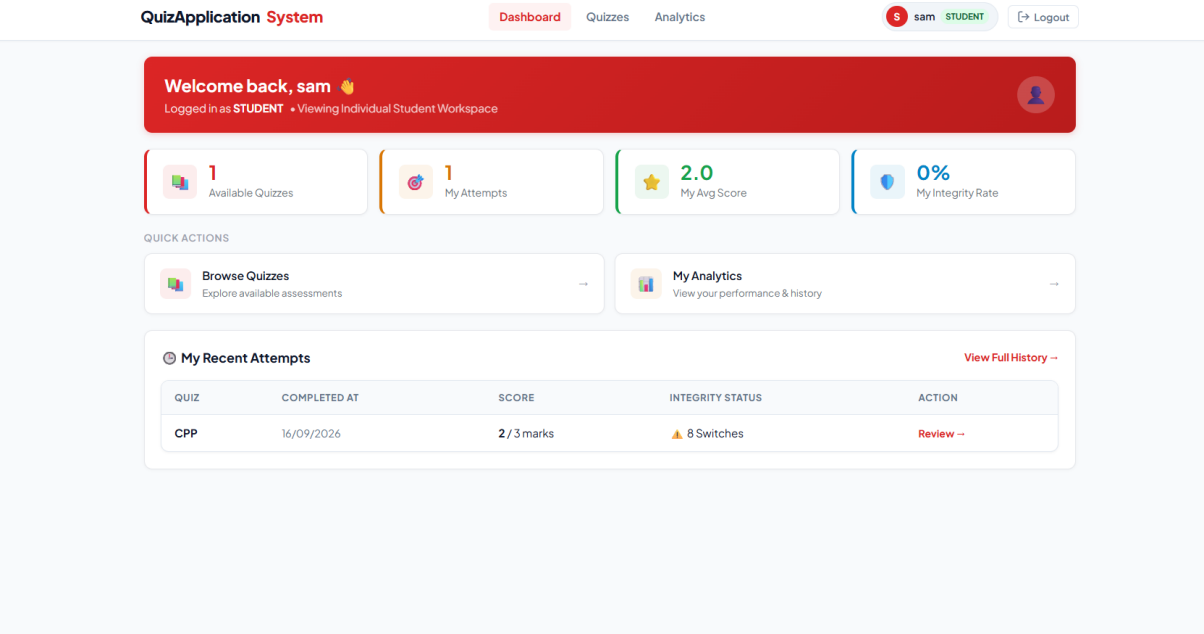


Figure 1.14 Student Dashboard

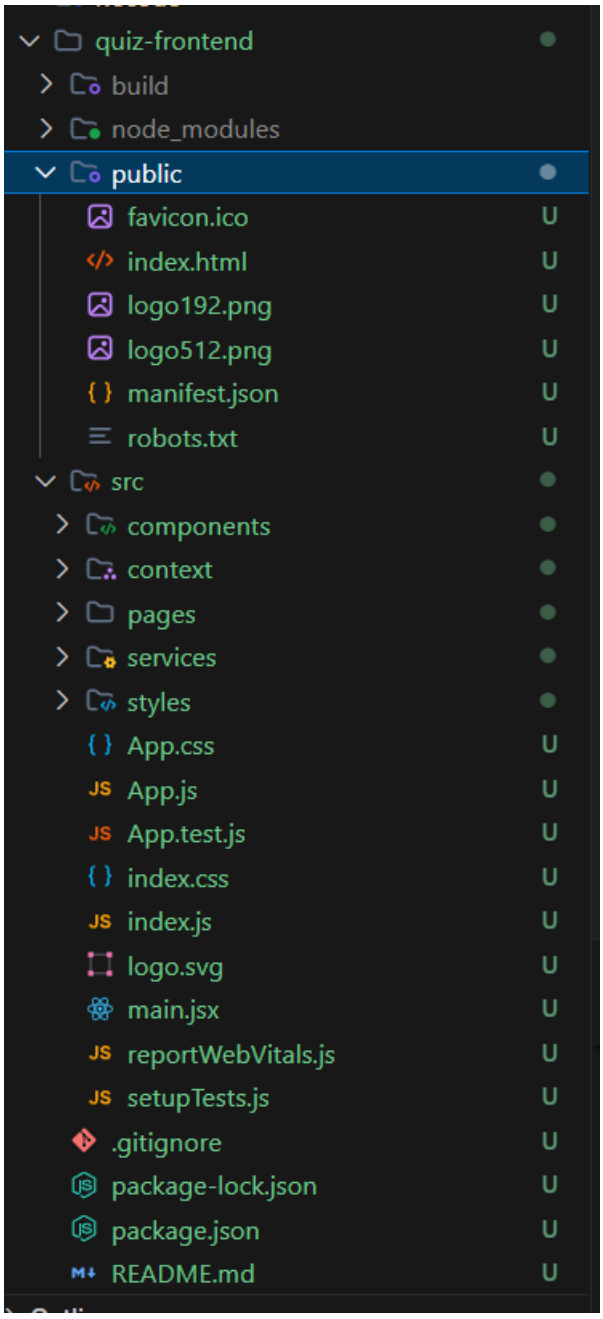


Fig 1.15 Frontend Folder Structure

6. TECHNICAL DESIGN

6.1 Performance and Reliability Targets

Metric Description	Target Threshold
Quiz Page Response Time	< 500ms for 95% of requests
User Authentication Response	< 1 second
Quiz/Question Loading Time	< 1 second
Quiz Submission Processing	< 2 seconds for 90% of submissions
Automated Evaluation	< 2 seconds for standard quizzes
Result Generation	< 2 seconds
Analytics Generation	< 3 seconds for complex queries
Database Query Response	< 1 second for 95% of requests

6.2 REST API Reference

6.2.1 Quiz Management

HTTP Method	API Endpoint	Description
POST	/api/quizzes	Create and initialize a new quiz with the required quiz details.
GET	/api/quizzes	Retrieve the list of available quizzes.
GET	/api/quizzes/{id}	Retrieve the details of a specific quiz.
PUT	/api/quizzes/{id}	Update the configuration and details of an existing quiz.
DELETE	/api/quizzes/{id}	Delete an existing quiz from the system.
POST	/api/bots/{id}/deploy	Execute the deployment workflow targeting designated conversational channels.
POST	/api/bots/{id}/train	Run the training pipeline to update intent models using current text datasets.

6.2.2 Question Management

HTTP Method	API Endpoint	Description
POST	/api/questions	Add a new question with MCQ options, correct answer, question type, and marks.
GET	/api/questions	Retrieve all questions available in the system.
GET	/api/questions/{id}	Retrieve details of a specific question.
GET	/api/questions/quiz/{quizId}	Retrieve all questions associated with a specific quiz.
PUT	/api/questions/{id}	Update the details of an existing question.
DELETE	/api/questions/{id}	Delete a question from the system.

6.2.3 Quiz Attempt and Response Management

HTTP Method	API Endpoint	Description
POST	/api/quiz-attempts	Start and create a new quiz attempt for a student.
GET	/api/quiz-attempts	Retrieve all quiz attempts recorded in the system.
GET	/api/quiz-attempts/{id}	Retrieve details of a specific quiz attempt.
GET	/api/quiz-attempts/user/{userId}	Retrieve quiz attempts associated with a specific student.
PUT	/api/quiz-attempts/{id}	Update the status, score, and completion details of a quiz attempt.

6.2.4 User Authentication and Administration

HTTP Method	API Endpoint	Description
POST	/api/auth/register	Register a new user in the Quiz Management System.
POST	/api/auth/login	Authenticate a user and generate a JWT authentication token.
GET	/api/admin/users	Retrieve all registered users for administrative management.
GET	/api/admin/users/{id}	Retrieve details of a specific user.
PUT	/api/admin/users/{id}	Update user information or role details.

6.2.5 Performance and Platform Analytics

HTTP Method	API Endpoint	Description
GET	/api/analytics	Retrieve all quiz and student performance analytics.
GET	/api/analytics/{id}	Retrieve analytics details for a specific record.
GET	/api/analytics/quiz/{quizId}	Retrieve performance analytics for a specific quiz.
GET	/api/analytics/user/{userId}	Retrieve performance analytics for a specific student.
POST	/api/analytics	Create and store a new analytics record.
PUT	/api/analytics/{id}	Update an existing analytics record.
DELETE	/api/analytics/{id}	Delete a specific analytics record.

7. TESTING

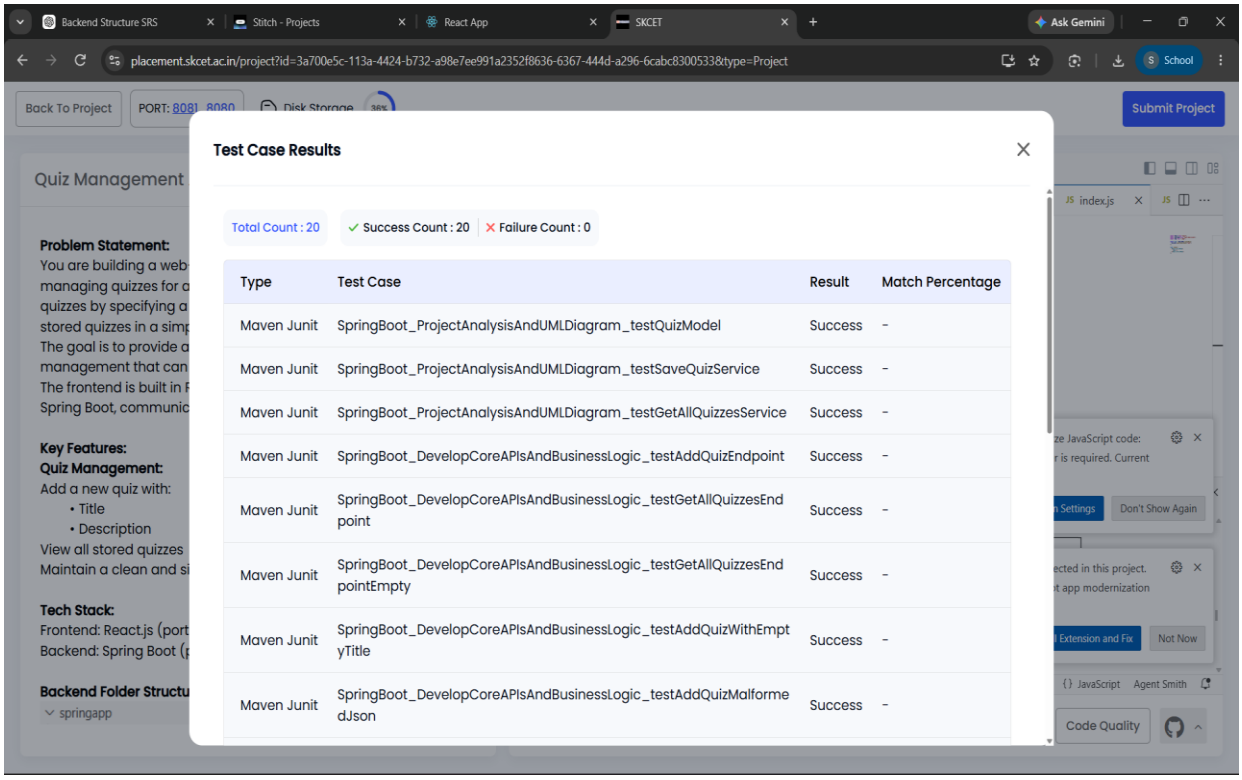


Fig 1.16 Test Cases

8. CONCLUSION & FUTURE ENCHANCEMENT

8.1 Project Summary

The **Quiz Application Management System** successfully demonstrates the effectiveness of a modular and secure web-based assessment architecture. By combining a robust **Spring Boot backend** with an intuitive **React.js frontend**, the system enables efficient quiz creation, question management, online assessment, automated evaluation, and result generation. The integration of **JWT authentication and role-based access control (RBAC)** ensures secure access for Administrators, Educators, and Students, while **analytics and gradebook management** provide centralized performance tracking. Overall, the system improves assessment efficiency, reduces manual evaluation effort, and provides a reliable platform for managing online quizzes.

8.2 Technical Constraints and Rationale

The architecture defines specific environmental requirements to maintain system reliability and compatibility. **Port 8080** is used for the Spring Boot backend, while the **React.js frontend** runs on **Port 3000** supporting smooth communication between the frontend and backend. The system requires **Java 17 or higher** to support the Spring Boot application and its modern framework features. These configuration requirements provide a stable environment for handling quiz creation, question management, student assessments, automated evaluation, and performance tracking.

8.3 Future Enhancements

- **Advanced Features:** Real-time updates, multimedia questions, AI-powered question generation, notifications, and multi-language support.
- **Scalability & Accessibility:** Cloud deployment, advanced analytics, mobile applications, and improved reporting.
- **Security:** End-to-end encryption and stronger authentication mechanisms for protecting user and assessment data.
- **Assessment Improvements:** Support for advanced question types, adaptive quizzes, and personalized learning paths.
- **System Integration:** Integration with Learning Management Systems (LMS), email services, and other educational platforms.

9. Appendix: Role-Based Permission Matrix

Platform Functionality	Student	Educator	Administrator	Platform Functionality	Student	Educator	Administrator	Platform Functionality
Register / Login	✓	✓	✓	Register / Login	✓	✓	✓	Register / Login
View Available Quizzes	✓	✓	✓	View Available Quizzes	✓	✓	✓	View Available Quizzes
Take Quiz	✓	-	-	Take Quiz	✓	-	-	Take Quiz
Submit Quiz	✓	-	-	Submit Quiz	✓	-	-	Submit Quiz
View Results	✓	✓	✓	View Results	✓	✓	✓	View Results
Create Quizzes	-	✓	✓	Create Quizzes	-	✓	✓	Create Quizzes
Manage Quizzes	-	✓	✓	Manage Quizzes	-	✓	✓	Manage Quizzes
Add / Manage Questions	-	✓	✓	Add / Manage Questions	-	✓	✓	Add / Manage Questions
Automated Evaluation	✓	✓	✓	Automated Evaluation	✓	✓	✓	Automated Evaluation
View Analytics	✓	✓	✓	View Analytics	✓	✓	✓	View Analytics
View Gradebook	✓	✓	✓	View Gradebook	✓	✓	✓	View Gradebook